

Из папки — в интернет.

До этого этапа ваши проекты жили только на вашем компьютере. Сегодня учим Claude публиковать их в интернет — через git, GitHub и worktrees. Дашборды из этапов 02–04 получают живые ссылки.

ДЛИТЕЛЬНОСТЬ

4–5 дней • 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

git • GitHub • GitHub Pages • worktrees • защита секретов

ГЛАВНЫЙ ИТОГ

ваши работы в интернете, открываются по ссылке

АРТЕФАКТ

опубликованный портфолио и дашборд бюджета по живой ссылке

Аккаунт GitHub и программа git – обязательно. Один skill для дисциплины worktrees – рекомендуется. Claude всё делает за вас, но аккаунт и программа должны быть.

ОБЯЗАТЕЛЬНО • ПРОГРАММА

git

Что это: программа для «контроля версий» – ведёт историю всех изменений в вашем проекте. Это та самая «машина времени», к которой обращается Claude, когда вы говорите «верни как было».

Зачем сейчас: без git ничего на этом этапе не получится – и ваш проект не попадёт на GitHub.

Где взять: git-scm.com/downloads. Установка стандартная, на всех вопросах оставляйте «по умолчанию».

Проверить: в терминале `git --version` должно вывести номер версии.

ОБЯЗАТЕЛЬНО • АККАУНТ

GitHub

Что это: крупнейший в мире «склад» проектов на git. Вы храните свои работы там, делитесь по ссылкам, бесплатно публикуете статичные сайты через GitHub Pages.

Зачем сейчас: чтобы ваш дашборд получил живой URL вида

`https://<username>.github.io/<project>/` – и его можно было показать другу или работодателю.

Где взять: github.com/signup. Регистрация бесплатная.

Проверить: вы можете войти на github.com и увидеть свою страницу профиля.

SSH-ключ или GitHub CLI (gh)

Что это: способ для вашего компьютера «доказать GitHub-у», что это вы. Без этого нельзя пушить (загружать) проекты.

Простой способ: поставьте GitHub CLI (cli.github.com) и один раз выполните `gh auth login` — он сам всё настроит.

Альтернатива: сгенерировать SSH-ключ и добавить в GitHub. Если потеряете маршрут — попросите Claude: *«проведи меня через настройку SSH-ключа для GitHub»*.

Проверить: `gh auth status` или `ssh -T git@github.com` — ответ должен подтвердить ваш логин.

Skill superpowers:using-git-worktrees

Что это: skill, который учит Claude правильно использовать git worktrees — «параллельные копии репозитория». Удобно, когда вы хотите работать над двумя ветками одновременно (например, доработка дашборда + эксперимент с тёмной темой) и не путаться в состояниях.

Зачем сейчас: без skill можно, но skill экономит время на типовых ошибках.

Проверить, не стоит ли: `/skills` — `superpowers:using-git-worktrees` в списке. Если ставили `superpowers` в этапе 04 — уже есть.

```
# внутри claude (slash-команда, не shell). Пакет тот же, что в этапе 04
/plugin install superpowers@claude-plugins-official
```

Все базовые операции git — init, commit, push, pull, branch — Claude делает сам. Вам не нужно их запоминать. Но в § I мы разберём, что они означают, чтобы вы могли осмысленно сказать «закоммить и запушь».

Git как «машина времени» для проекта

Git — это программа, которая *фиксирует* состояние вашего проекта в любой момент. Каждое сохранение называется коммит — это «фотография» состояния всех файлов.

Что это даёт:

- Можете в любой момент *откатиться* к старой версии: «верни как было два дня назад».
- Видите *историю*: что и когда менялось, кем (если работали с кем-то).
- Можете *экспериментировать*: создаёте «параллельную линию» (ветку), пробуете что-то, и если не понравилось — просто отбрасываете.
- Можете *делиться*: загружаете на GitHub — и у всех есть полная история проекта.

Аналогия: «Версии файлов» в macOS или «Журнал версий» в Google Docs — но для всего проекта целиком, и на стероидах.

Минимум команд (которые Claude использует за вас):

```
# создать репозиторий в текущей папке
$ git init

# сохранить состояние (коммит)
$ git add .
$ git commit -m "первый коммит"

# отправить на GitHub
$ git push
```

Запоминать их не надо. Достаточно понимать *смысл*: **добавить** в индекс → **зафиксировать** как коммит → **отправить** на удалённый.

Репозиторий, ветки и удалённый

Репозиторий — это папка, в которой git ведёт историю. Внутри есть скрытая папка `.git` — именно там лежат все коммиты.

Ветка — это параллельная линия истории. Главная называется `main` (раньше — `master`). Когда хотите попробовать что-то рискованное — создаёте новую ветку (например, `experiment/dark-theme`), коммитите туда, и если получилось — *сливаете* (merge) обратно в `main`. Если не получилось — просто удаляете ветку.

Удалённый (remote) — это копия вашего репозитория где-то на сервере. Обычно — на GitHub. Команда `push` отправляет ваши коммиты туда, `pull` — забирает изменения оттуда.

Типичный диалог:

```
> создай в этой папке git-репозиторий, сделай первый коммит «init».  
> создай ветку experiment/dark-theme.  
> в ней сделай dark.html (тёмная версия моей страницы).  
> покажи изменения.  
> если выглядит нормально — слей в main и удали ветку.
```

GitHub Pages: бесплатный хостинг для ваших страниц

GitHub Pages — это «магия»: загружаете HTML/CSS/JS в репозиторий на GitHub, включаете одну галочку в настройках — и через минуту ваш сайт доступен по адресу `https://<username>.github.io/<repo>/`.

Что подходит:

- статичные сайты без бэкенда (всё, что мы делали в этапах 00–04);
- портфолио, лендинги, дашборды на CSV-данных;
- демо ваших калькуляторов и игр.

Что не подходит:

- сайты с серверной логикой (этап 06 будет про них — через Vercel или Railway);

- проекты с секретами, которые требуют живой работы с API (вызовы к Alpha Vantage с ключом нельзя делать прямо в браузере).

Полный диалог публикации:

```
> создай на github новый публичный репозиторий "fi-forecast".  
> запусти туда текущий проект.  
> включи GitHub Pages с источником main / root.  
> дай мне публичную ссылку и проверь через playwright,  
> что страница открывается и не сломалась.
```

Через минуту – ссылка вида `https://yourname.github.io/fi-forecast/`, которую можно показать кому угодно.

README и .gitignore: «обложка» и «не показывать»

README.md – первое, что видит человек, открывший ваш репозиторий на GitHub. Без него странно, как лекция без названия.

Минимум для хорошего README:

1. **Заголовок и одна строка о чём проект.**
2. **Демо-ссылка** – на GitHub Pages или скриншот.
3. **Что это умеет** – 3–5 пунктов.
4. **Как запустить локально** – 2–3 команды.
5. **Откуда данные / какие нужны секреты** – если применимо.
6. **Лицензия** – MIT по умолчанию хорошо.

Claude составит README за вас одной командой:

```
> составь README.md для этого проекта.  
> опиши: что делает, как запустить, демо-ссылка  
> (https://...github.io/...), какие нужны секреты,  
> лицензия MIT. сделай двуязычным: ru + en.
```

.gitignore – список «что git не должен видеть»: `.env`, `data/private/`, `node_modules/`, `.DS_Store`. Без него можно случайно опубликовать API-ключ в интернет – и через час кто-то будет тратить ваши деньги через ваш аккаунт.

```
> составь .gitignore для этого проекта.  
> обязательно: .env, data/personal/, *.log,  
> макросные файлы macOS/Windows.  
> добавь раздел "private" с моими личными исключениями.
```

Защита секретов: что нельзя коммитить

Самая дорогая ошибка непрофессионала – случайно отправить API-ключ или пароль в публичный репозиторий. Боты сканируют GitHub каждые несколько минут – ключ найдут за часы.

Чего НЕ делать никогда:

- не писать ключи прямо в HTML/JS-файлах;
- не коммитить `.env`;
- не класть пароли в `CLAUDE.md`;
- не коммитить дампы баз с личными данными;
- не копировать SSH-ключ в репо.

Как защититься:

1. `.gitignore` с `.env`, `secrets/`, `data/private/`;
2. в коде – всегда читаем секреты из окружения (`process.env.X`, `os.getenv("X")`);
3. в README – пример `.env` без значений (`.env.example`);
4. перед каждым коммитом – «покажи *git diff*, проверь, не утекают ли секреты»;
5. регулярные сканы – через hooks (этап 08a) или внешние сервисы (например, gitleaks).

Дисциплина:

```
> перед тем как пушить, прогон проверки:  
> git diff --staged | grep -iE "key|secret|token|password"  
> если что-то нашлось – откати и поправь, перед коммитом не пускай.
```

Если случайно опубликовали – не тяните: *отозвать* ключ через сервис, поменять пароль, и только потом думать про «как удалить из истории git» (это сложнее, чем кажется).

Worktrees: параллельные ветки без путаницы

Обычная работа с ветками – «переключайся туда-сюда»: `git checkout main`, потом `git checkout experiment/dark-theme`. Состояние папки *меняется*: одни и те же файлы превращаются в разные версии.

Worktrees – альтернатива: каждая ветка живёт в *своей папке*. Не надо переключаться – вы просто открываете другую папку.

Когда полезно:

- Хотите делать *две вещи параллельно* – например, исправить баг в main и дописать фичу в experiment;
- Запускаете *агента-стажёра* в другой ветке (этап 07 – subagents часто работают в worktree, чтобы не мешать вам);
- Не хотите случайно потерять незакоммиченные изменения при переключении.

Команды (Claude делает за вас):

```
# создать worktree в папке-сестре
$ git worktree add ../my-project-dark experiment/dark-theme

# список всех worktrees
$ git worktree list

# удалить worktree, когда не нужен
$ git worktree remove ../my-project-dark
```

Skill `superpowers:using-git-worktrees` добавляет дисциплину – Claude правильно создаёт worktree и убирает за собой.

Форки и Pull Requests — на будущее

На этом этапе мы работаем со своими репозиториями. Но стоит знать два понятия для будущего:

- **Форк** – кнопка «Fork» на чужом репо. Вы получаете свою копию, в которой можете делать что угодно. Полезно, если хотите доработать чужую тему сайта или взять чей-то проект как основу.
- **Pull Request (PR)** – способ *предложить* владельцу оригинального репозитория ваши изменения. Сделали форк, исправили опечатку в README, послали PR – владелец смотрит и либо принимает, либо отклоняет.

Зачем это нужно сейчас знать: на капстоуне, возможно, будете публиковать собственный плагин для Claude Code. Когда другие пользователи захотят вам помочь – они сделают форк и PR. Через GitHub CLI это работает за 30 секунд.

PRACTICUM

Прогон • из папки в интернет за 15 минут

1. Возьмите папку `fi-forecast` из этапа 04. Запустите там `claude`.
2. Промпт:

```
> сделай git-репозиторий в этой папке (если ещё нет).  
> составь .gitignore: .env, data/private/, *.log,  
> .DS_Store, Thumbs.db, screenshots/.  
> добавь все файлы, кроме игнорируемых, в первый коммит «init: fi-forecast».  
> покажи git status и git log – должен быть один чистый коммит.
```

3. Промпт:

```
> через gh создай новый ПУБЛИЧНЫЙ репозиторий fi-forecast  
> и запусти туда текущую ветку main.  
> включи GitHub Pages, источник: main / root.  
> через playwright открой получившуюся ссылку  
> (через 30-60 секунд после включения)  
> и убедись, что страница загружается.  
> верни мне URL.
```

4. Откройте URL в своём браузере. Поделитесь ссылкой с другом – пусть откроет с телефона.
5. Промпт на README:

```
> составь README.md по моему проекту – двуязычный (ru + en).  
> разделы: что это, демо-ссылка, как запустить локально,  
> используемые технологии (без сборщиков, chart.js через CDN),  
> параметры (входы формы), лицензия MIT.  
> закоммить и запусть, чтобы README появился на странице репо.
```

6. Через минуту откройте репо на github.com – должен появиться README с форматированием.

7. Тренировка отката: сделайте небольшое неудачное изменение в `спес.md`, закоммитьте. Потом скажите Claude:

```
> верни спец.md к состоянию предыдущего коммита (до моего «неудачного» изменения)
> покажи diff. если ОК — закоммить «revert: undo bad спец edit».
```

8. Поэкспериментируйте с worktrees:

```
> через git worktree создай папку-сестру fi-forecast-dark
> на новой ветке experiment/dark-theme.
> открой её в проводнике — это та же история, но другая ветка.
> не переключаясь, сделай в ней dark-вариант страницы.
> закоммить туда. потом удали worktree, когда наиграешься.
```

Что мы тренируем: **полный цикл публикации** — **init** → **коммит** → **GitHub** → **Pages** → **README** → **откат** → **worktree**. Дальше делаем большие проекты на этом фундаменте.

ПРОЕКТ А · ПО ИНСТРУКЦИИ

GitHub-портфолио со всеми вашими работами

Цель: один публичный репозиторий `portfolio` с `index.html`, который ведёт на ссылки ваших проектов с этапов 00–04. Вид – как авторская страница, не типовой шаблон.

Шаги

1. Сначала опубликуйте на GitHub Pages каждый проект, который хотите показать (карточка обо мне, книжная полка, калькулятор процента, угадай число, дашборд портфеля, прогноз FI). Промпт по каждому – как в «Прогоне» выше.
2. Создайте папку `portfolio`, запустите `claude`.
3. Промпт:

```
> используя skill frontend-design, сделай index.html для моего портфолио.  
> стиль – авторская editorial-страница, как старый журнал работ.  
> структура:  
>   – hero: имя, одна фраза «кто я и что делаю»;  
>   – список 6–8 проектов: миниатюра, название, одна-две строки,  
>     ссылка на live-демо (URL я тебе дам).  
>   – футер с github и (опционально) линкедин.  
> миниатюры – пусть claude сгенерирует svg или возьмёт скриншот  
> через playwright (открой каждый проект, screenshot 800x600).
```

4. Дайте Claude список ваших URL-ов (по одному на строке).
5. Откройте через локальный сервер. Уточняйте: «сделай миниатюры аккуратнее, без обрезки», «группируй проекты: финансы / игры / прочее».
6. Когда нравится – `init` / `commit` / `push` / Pages, как в прогоне.

7. Финал – через playwright проверьте на нескольких viewport-ах:

```
> через playwright открой моё портфолио (URL: ...)
> в трёх размерах: 1440x900, 768x1024, 390x844.
> для каждого – скриншот и вердикт «нет ли проблем».
```

8. Опубликуйте URL у себя в соцсетях, в подписи к email, в CV. Это и есть ваш «AI-нативный портфолио».

Что вы здесь освоили

- публикацию каждого проекта на GitHub Pages по шаблону;
- агрегацию ссылок в портфолио-страницу с «своим лицом»;
- проверку результата через playwright на нескольких устройствах.

Дашборд бюджета на демо-данных с двуязычным README

Цель: публикация дашборда из этапа 02 – но на демо-данных (без вашей реальной выписки) и с качественным README. Цель – иметь публичный «showcase», который можно показать.

Шаги

1. Создайте новую папку `budget-dashboard-demo`. Скопируйте туда `dashboard.html` и `CLAUDE.md` из `portfolio-journal`. Реальные данные не копируйте.

2. Промпт:

```
> сгенерируй data/operations.csv с 80 реалистичными
> инвестиционными операциями за 18 месяцев.
> шесть тикеров, buy/sell/dividend смешаны. фейковые имена,
> никаких персональных данных, чтобы можно было опубликовать.
```

3. Откройте dashboard локально. Убедитесь, что работает на демо-данных.

4. Промпт:

```
> составь CLAUDE.md и .gitignore.
> в .gitignore обязательно: .env, data/personal/, screenshots/.
> составь README.md, двуязычный (ru + en) с разделами:
> – заголовок и одна строка о проекте;
> – демо-ссылка (заглушка, обновлю после деплоя);
> – что показывает дашборд (3-4 пункта);
> – формат входных данных (схема CSV);
> – disclaimer: "данные – синтетические, для демо";
> – лицензия MIT.
```

5. `git init / commit / push` на GitHub как публичный, включите Pages.

6. Замените заглушку демо-ссылки в README на настоящую, ещё раз закоммить.

7. Финальная ревизия через playwright + аудит:

```
> через playwright открой опубликованный дашборд.  
> пройди три checks:  
> – все ли числа отображаются;  
> – на телефоне (390x844) ничего не сломано;  
> – README на github.com/(...) рендерится корректно (через WebFetch).  
> покажи вердикт.
```

8. Поделитесь URL с другом и послушайте обратную связь.

Что вы здесь освоили

- «фейковые данные для публичного demo» – типовой подход, когда исходные данные приватные;
- двуязычный README со всеми разделами, по которым другие поймут проект за 30 секунд;
- цикл «публикация → проверка → правка README» в нескольких коммитах.

ПРОЕКТ С • САМОСТОЯТЕЛЬНО

Опубликовать любой свой проект из этапов 03–04

Цель: пройти полный цикл «папка → GitHub Pages → README → live URL» для любого вашего проекта (трекер котировок, прогноз сбережений, симулятор пенсии, дайджест новостей).

Подсказки

- Если в проекте есть API-ключи – используйте «демо-режим»: сгенерированные данные вместо живых API. Иначе ключ утечёт.
- README обязателен, двуязычный, с демо-ссылкой и скриншотом.
- Через playwright проверьте на трёх viewport-ах после деплоя.
- В финале – пришлите ссылку другу, попросите 30-секундное мнение.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ проект опубликован публично, есть рабочая ссылка вида `https://<you>.github.io/<name>/` ;
- ✓ в репо есть `README.md` (двуязычный) и `.gitignore` ;
- ✓ в коммитах нет ни одного API-ключа, пароля или личной выписки (проверьте через `git log -p | grep -iE "key|secret"`);
- ✓ README на странице репо рендерится с демо-ссылкой и скриншотом;
- ✓ через playwright проверены 1440, 768, 390 viewport;
- ✓ друг открыл ссылку с телефона – работает.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Прогуляйся по моему репо как HR-специалист, оценивающий портфолио кандидата. За что бы ты похвалил, что зацепил бы как «зелёный флаг»? Что выглядит «совсем сыро» и стоит подтянуть?»

Публичный «портфель ссылок» на все ваши работы

Цель: вариация проекта A, но с другим жанром – не «авторская editorial-страница», а каталог ссылок: компактный, плотный, под один скролл, как «links.dev»-страницы. Для тех, кто предпочитает не «дизайн ради дизайна», а «дайте список с живыми ссылками».

Подсказки

- Структура та же, что в A – репо, Pages, README. Меняется только дизайн.
- В промпте укажите: «таблица или плотная сетка ссылок, без иллюстраций, как linktree но с характером».
- Сгруппируйте ссылки: «финансовые проекты», «игры», «учебные», «эксперименты».
- Используйте worktree, если хотите параллельно держать оба варианта (editorial + directory) на разных ветках.
- На капстоуне сможете решить, какой вариант – ваш «личный сайт».

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ есть отдельный репо `links` со своим README;
- ✓ страница помещается в один скролл на десктопе и 3–4 скrolла на телефоне;
- ✓ ссылки сгруппированы (минимум 3 группы);
- ✓ дизайн узнаваем, не стандартный bootstrap-шаблон;
- ✓ через playwright проведена ревизия на трёх viewport;
- ✓ попробовали worktree: либо альтернативная ветка с другим стилем, либо просто проба.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Сравни мои два варианта (editorial из проекта A и directory из проекта D). Для каких ситуаций каждый лучше? Если бы ты вёл блог по ИИ – который выбрал бы как «главную страницу»? Почему?»

Git	Программа контроля версий. «Машина времени» для проекта – фиксирует каждое изменение, позволяет откатываться, разделять историю на ветки и делиться.
Репозиторий	Папка проекта, которую отслеживает git. Внутри – скрытая <code>.git</code> с историей всех изменений.
Коммит	«Фотография» проекта в момент сохранения. Имеет автора, дату и описание. Можно вернуться к любому коммиту в истории.
Ветка	Параллельная линия истории. Главная – <code>main</code> . На экспериментальные ветки делают рискованные изменения, потом сливают в <code>main</code> или отбрасывают.
Удалённый (remote)	Копия вашего репозитория на стороннем сервере (обычно GitHub). <code>git push</code> отправляет туда коммиты, <code>git pull</code> забирает изменения оттуда.
GitHub Pages	Бесплатный хостинг от GitHub: загрузил HTML/CSS/JS в репозиторий, включил галочку – и сайт доступен по адресу <code>https://<username>.github.io/<repo>/</code> .
README.md	Главная «обложка» репозитория. Markdown-файл в корне. GitHub показывает его на странице репо. Туда – что это, как запустить, демо-ссылка, скриншоты, лицензия.
.gitignore	Файл со списком того, что git должен игнорировать: <code>.env</code> , секреты, кэши, временные файлы, <code>node_modules/</code> . Без него легко случайно опубликовать API-ключ.
Worktree	Изолированная копия репозитория в отдельной папке – для параллельной работы над несколькими ветками одновременно. Создаётся через <code>git worktree add</code> , удаляется через <code>git worktree remove</code> .
Форк	Ваша личная копия чужого репозитория на GitHub. Делается одной кнопкой. Вы можете в ней что угодно – оригинал не трогается.
Pull Request (PR)	Способ предложить владельцу другого репозитория ваши изменения. Сделали форк, поправили, послали PR – владелец смотрит, принимает или отклоняет.

gh (GitHub CLI)

Командная утилита от GitHub: создавать репозитории, делать PR, смотреть issues – всё из терминала. Claude использует gh для большинства действий с GitHub.

01 Чем коммит отличается от «сохранить» в Word?

«Сохранить» в Word перезаписывает файл – старая версия теряется. Коммит – «фотография» проекта в момент сохранения, со всеми файлами, автором, датой и описанием. Все коммиты остаются в истории, можно вернуться к любому.

02 Что точно нельзя коммитить в публичный репозиторий?

API-ключи, пароли, токены, дампы баз с личными данными, личные финансовые выписки, SSH-ключи. Всё это идёт в `.env` и `.gitignore`. Боты сканируют GitHub каждые несколько минут – утёкший ключ найдут за часы.

03 Что такое GitHub Pages и для каких проектов он подходит?

Бесплатный хостинг для статичных сайтов. Подходит: HTML/CSS/JS без серверной логики, портфолио, дашборды на CSV, лендинги, демо. Не подходит: проекты с серверным кодом, проекты, которые должны делать API-вызовы с секретным ключом (для них – Vercel/Railway, этап 06).

04 Зачем нужен **worktree**, если есть обычный `git checkout`?

Чтобы работать над двумя ветками *одновременно*, не переключаясь. Каждый **worktree** – своя папка. Не нужно прятать незакоммиченные изменения, не нужно ждать, пока всё закоммитится. Особенно полезно для subagents (этап 07): агент работает в своей папке, не мешая вам.

05

Что обязательно должно быть в хорошем README?

Шесть пунктов: **заголовок и строка о чём проект**; **демо-ссылка или скриншот**; **что это умеет (3–5 пунктов)**; **как запустить локально**; **откуда данные / какие нужны секреты**; **лицензия**. Через 30 секунд чтения постороннему должно быть понятно, чем проект полезен.

06

Что делать, если случайно опубликовали API-ключ в публичный репо?

Сразу – **отозвать ключ** через сервис, выпустивший его. Поменять пароль если ключ был связан. Только потом думать про «удалить из истории git» (это сложнее, чем кажется – даже после `git rebase` копии могут остаться у ботов, скачавших ключ за первые минуты).

07

Какой минимум команд **git** нужно *понимать* (даже если их запускает Claude)?

`git init` (создать репо), `git add` (добавить в индекс), `git commit` (зафиксировать как коммит), `git push` (отправить на удалённый), `git pull` (забрать изменения), `git status` (что сейчас), `git log` (история), `git branch` (ветки), `git checkout` (переключиться).

Что я теперь умею

- ☐ Установлен git и есть аккаунт GitHub; `gh auth status` показывает мой логин.
 - ☐ Понимаю: репозиторий, коммит, ветка, удалённый, push/pull, merge.
 - ☐ Опубликовал хотя бы один проект на GitHub Pages с живой ссылкой.
 - ☐ Пишу хороший README (двухязычный, с демо-ссылкой и 6 разделами).
 - ☐ Знаю, что НЕ идёт в git (`.env` , ключи, личные данные); защищаюсь через `.gitignore` .
 - ☐ Попробовал worktree для параллельной работы над двумя ветками.
 - ☐ Сделал четыре проекта: GitHub-портфолио (editorial), демо-дашборд бюджета, опубликовал свой проект из этапов 03–04, портфель ссылок.
-
-

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

«Проект, который не опубликован — всё равно что не сделан. Опубликованный — работает на вас, даже когда вы спите».

Этап 05 пройден — ваши работы получили живые ссылки. Дальше учим Claude собирать полные веб-приложения с серверами, базами и авторизацией.

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 04
- Этап 06